Hash Function Distribution and Timing Lab

CMPS 2131 Data Structures | C23 | About 3 hours

VIDEO LINK https://youtu.be/z4xeBNSByxg?si=h1sWpg74_PzouE6n

In this lab, you will use the supplied C program to compare how several hash functions distribute integer and string keys into 65,536 buckets. You will also compare hash computation time for four string hash functions. The archive measures bucket occupancy and hash-function time; it does not contain a separate-chaining hash table implementation.

Learning goals

- Explain how a hash function and a bucket-reduction rule together produce a bucket index.
- Use bucket counts to compare observed distribution for different key sets.
- Explain why keys with regular patterns, such as aligned addresses, can expose weaknesses in a reduction rule.
- Distinguish distribution statistics from measured hash-computation time and explain the limits of each.

Files in the archive

File	Role in the lab
hash_functions.h	Function declarations and the universal_hash type.
hash_functions.c	Integer-key and string-key hash functions.
measure_hashes.c	Runs the distribution experiments and string-hash timing comparison.
test_hash_functions.c	Checks known outputs and verifies that results fall within requested bucket ranges.

Keep these four files together in one working directory. The source uses C23 syntax. Record your compiler version with gcc –version.

Compiler: gcc (GCC) 16.1.1 20260515 (Red Hat 16.1.1-2)

1. Compile and run

Compile the measurement program with optimization, then run it and save the complete output.

```
gcc -std=c23 -Wall -Wextra -Wpedantic -O2 measure_hashes.c hash_functions.c -o
measure_hashes
./measure_hashes
```

Compile and run the correctness checks separately.

```
gcc -std=c23 -Wall -Wextra -Wpedantic -O2 test_hash_functions.c hash_functions.c -o
test_hash_functions
./test_hash_functions
```

The expected test message is “all hash function tests passed”. These tests check selected outputs and range properties. They do not establish that a function distributes every possible key set well.

2. Understand the reported measures

For each distribution run, the program reports the table size m, the number of nonempty buckets (used), the largest bucket population (longest), and avg_cmp. Here avg_cmp is calculated from bucket populations as Σ[c(c + 1) / 2] / N, where c is the number of keys in a bucket and N is the number of keys.

Avg_cmp models the average successful search comparisons if each occupied bucket were represented as a separate chain and every stored key were searched once. The archive does not build those chains or time lookups. Treat avg_cmp as a distribution-derived comparison estimate, not as a measured lookup time.

The program also prints the load factor α = N / M and an idealized reference value 1 + α / 2. Use that as a comparison for the chaining estimate when the keys are distributed uniformly; it is not a guarantee for the particular key set.

The final section reports nanoseconds per key for hashing the 8-character student ID strings. That is a hash-computation microbenchmark, not a hash-table lookup benchmark. Results depend on the machine, compiler, and system activity.

3. Predict before reading the results

Inspect hash_functions.c and measure_hashes.c. Before running the program, predict where you expect each choice to distribute well or poorly. Give a reason based on the key pattern and the function or reduction rule.

Experiment	Choices to predict	Prediction and reason
A: integer student IDs	division by 65,536; division by 65,521; Fibonacci top 16 bits; fmix64 low 16 bits; universal	For consecutive numbers, such as student IDs, I expect few keys to collide because `% 65536` should spread the keys across buckets well. I expect similar behavior for division by 65,521 because the number of buckets is close to 65,536. For Fibonacci hashing, I expect a well distribution,because the keys would spread across different bucket positions, however collisions would still be possible. I expect fmix64 to distribute student IDs well because it mixes their bits before choosing a bucket, helping spread the keys out.
B: aligned addresses	same integer choices; universal is expected to report n/a	Division by 65,536: I expect poor distribution because the keys won't be well distributed, causing more collisions. Division by 65,521: I expect good distribution because the keys are not limited to buckets that are multiples of 16, creating fewer

		collisions. Fibonacci hashing: I expect good distribution because it helps more buckets get used. fmix64: I expect good distribution because mixing the address bits before choosing a bucket reduces the effect of addresses being multiples of 16, helping more buckets get used and spreading the keys more uniformly. Universal hashing: Universal hashing would report n/a because the keys are larger than 2,147,483,647, so the program skips this method.
C: student IDs as strings	character sum; poly31; djb2; FNV-1a	Character sum: I expect poor distribution because different IDs can contain the same characters, giving them the same hash. Poly31: I expect good distribution because the position of each digit affects the hash. Djb2: I expect good distribution because character order affects the hash, helping spread the IDs across buckets. FNV-1a: I expect good distribution because it mixes each character into the hash, helping spread the IDs across buckets.
D: item0 through item49999	character sum; poly31; djb2; FNV-1a	Character sum: I expect poor distribution because different names can have the same character sum, causing more collisions. Poly31: I expect good distribution because the position of each character affects the hash, helping spread the names across buckets. Djb2: I expect good distribution because character order affects the hash, helping different names get

		different hashes. FNV-1a: I expect good distribution because it mixes each character into the hash, helping spread the names across buckets.

. Experiment A: integer student IDs

Run the program and find section A. It creates 50,000 integer IDs using years 2019 through 2026, with 6,250 IDs per year. Compare these bucket-selection choices: modulo 65,536, modulo the nearby prime 65,521, Fibonacci hashing to 16 bits, fmix64 followed by taking the low 16 bits, and one selected universal hash function.

Copy the output values into the table. The program uses a fixed key set and fixed parameters, so distribution statistics should be repeatable. The universal hash parameters are selected from a deterministic pseudo-random generator in this program.

Rule	Buckets used	Longest bucket	Avg_cmp
k % 65,536	46,786	2	1.06
k % 65,521	46,771	2	1.06
Fibonacci, top 16 bits	46,363	2	1.07
fmix64, low 16 bits	34,968	7	1.38
Universal	34,230	2	1.32

Questions: Which choice used the most buckets? Which had the largest bucket population? How does the modulo-65,536 result compare with modulo 65,521? Use the ID structure and the code to explain the differences you observe.

The `% 65536` rule used the most buckets, with 46,786 buckets used.

The fmix64, low 16 bits rule had the largest bucket population, with 7 keys in one bucket.

I would describe the modulo results as similar because both rules spread consecutive student IDs across consecutive buckets, and the bucket counts are close. The `% 65536` rule used 46,786 buckets, while `% 65521` used 46,771. Both had a longest bucket of 2 and an avg_cmp of 1.06.

5. Experiment B: aligned integer addresses

Find section B. The program generates 50,000 values starting at 0x7f3a20000000 and adding 16 each time. These values resemble addresses with 16-byte alignment: their low four bits are always zero. Compare the same integer-hashing choices from Experiment A.

Rule	Buckets used	Longest bucket	Avg_cmp	Observation
k % 65,536	4,096	13	6.61	Keys crowded into fewer buckets
k % 65,521	50,000	1	1.00	No collisions for this key set
Fibonacci, top 16 bits	32,670	2	1.35	At most 2 keys per bucket

fmix64, low 16 bits	**34,985**	6	1.38	At most 6 keys per bucket
Universal	n/a	n/a	n/a	Keys exceed the supported range

The universal family is not applied to these address keys: the source checks whether keys are below $p = 2^{31} - 1$ and prints n/a when they are not. Explain why taking low bits after division by a power of two interacts with 16-byte alignment. Compare that result with the other methods, using your measurements.

The program output n/a for universal hashing because the keys were too big for the program. Key sizes must be less than 2,147,483,647.

I expected higher collisions with % 65536 because 50,000 keys are mapped to 4,096 buckets. Hash values are all multiples of 16, hash values in binary all end in four 0s. The hash value % 65,536, given 65,536 is a power of 2, retains the 16 least significant digits in binary, including the four 0s. Therefore, fewer buckets are used, and many keys map to the same buckets.

The % 65536 rule had a maximum bucket size of 13 and an average comparison count of 6.61 for address keys. The % 65521 rule performed better for address keys, with a maximum bucket size of 1 and an avg_cmp of 1.00, for address keys, no collisions occurred.

Both Fibonacci and fmix64 are expected to make fewer searches than % 65536. The expected number of searches for Fibonacci and fmix64 is 1. 35 and 1. 38, respectively, compared with 6. 61 for % 65536. .

6. Experiments C and D: string keys

In section C, the program formats the same student IDs as 8-character strings. In section D, it uses the strings item0 through item49999. For both sets, it applies the character-sum hash, poly31, djb2, and FNV-1a, then reduces each 64-bit or 32-bit hash value modulo 65,536.

String key set	Hash function	Buckets used	Longest bucket	Avg_cmp
Student ID strings	Character sum	41	3,330	1,200.94
Student ID strings	poly31	27,064	4	1.68
Student ID strings	djb2	25,068	5	1.78
Student ID strings	FNV-1a	33,924	7	1.42
item0 to item49999	Character sum	131	2,623	798.25
item0 to item49999	poly31	27,210	6	1.79
item0 to item49999	djb2	26,758	6	1.81
item0 to item49999	FNV-1a	33,382	7	1.44

Questions: Why does the character-sum function ignore character order? Find two different strings or describe a pair of rearrangements that collide under a character sum. Which functions distribute each supplied set better? Did the same function rank similarly for both key sets? Explain using the key patterns and recorded values.

Character sum ignores the order of characters because adding same characters in different order doesn't change the hash produced. For example, "ab" and "ba" collide because they both produce the sum of 195.

FNV-1a produced a better distribution based on avg_cmp. It had the lowest values for both key sets: 1.42 for student IDs and 1.44 for item names.

The functions had the same rankings for both key sets using avg_cmp. FNV-1a is ranked as the top function, followed by poly31 and thirdly djb2. Character sum was the worst-performing function, with its avg_cmp values of 1,200.94 for student IDs and 798.25 for item names.

The student IDs share year prefixes within each year group, and all item names start with "item". Character sum adds the character values in a string, so many different strings can have the same sum. FNV-1a mixes the characters and their order, helping spread these keys across more buckets.

7. Experiment E: hash computation time

At the end of the output, record the nanoseconds per key for the four string functions. The program hashes the 8-character student ID strings 200 times across 50,000 keys for each function.

Hash function	Reported ns per key
Character sum	4.14
poly31	3.92
djb2	4.56
FNV-1a	3.81

Run the program five times and calculate the median for each function. Keep all raw results. Compare the speed ranking with the distribution results for student ID strings. A faster hash computation does not necessarily produce better distribution, and this measurement does not include bucket lookup or chain traversal.

Timing limitation: measure_hashes.c uses timespec_get with TIME_UTC. This is a wall-clock source, so small measurements may be affected by scheduling and clock adjustments. Treat small differences cautiously; report the compiler, machine, repeated values, and median.

Hash function	Run 1	Run 2	Run 3	Run 4	Run 5	Median
Character sum	4.14	5.91	6.04	5.82	6.09	**5.91**
poly31	3.92	5.70	5.38	5.43	5.38	**5.38**
djb2	4.56	5.39	6.37	6.35	6.33	**6.33**
FNV-1a	3.81	4.73	5.31	5.36	5.33	**5.31**

machine_details.

Compiler: GCC 16.1.1
Operating system: Fedora
Architecture: x86_64
Processor: AMD Ryzen 7 PRO 6850U with Radeon Graphics
Logical CPUs: 16

8. Evidence and submission

- Completed predictions written before examining the output.
- Compiler version, exact compile commands, and complete program output.
- Completed distribution tables for integer IDs, aligned addresses, and both string key sets.
- Five timing runs, the raw values, and the median for each string hash function
- One labeled grouped bar graph comparing avg_cmp for the five integer-hashing choices on student IDs and aligned addresses.
- One labeled graph comparing avg_cmp for the four string hash functions across the two string key sets.
- Written explanations that refer to specific values and distinguish distribution-derived avg_cmp from measured nanoseconds per key.
- A 3–5 minute walkthrough showing the run, one graph, and an explanation supported by measurements.

Conclusion

Write 150–250 words answering this question: How did the hash function and the structure of the keys affect bucket distribution in the supplied experiments, and how did hash computation time compare? Support the explanation with at least four specific measurements from at least two key sets. State one limitation of the measurements.

Evidence rule: avoid saying a hash function is "good" or "fast" without identifying the key set and measurement that support the claim. Separate what you observed from what you infer about other possible keys.

The experimental results showed differences in the number of buckets used for different types of keys and hash functions. For the student ID keys, k % 65,536 used 46,786 buckets and produced an avg_cmp of 1.06. When the same hash function was used with 16-byte-aligned addresses, it used only 4,096 buckets.. The longest bucket contained 13 keys, and the avg_cmp was 6.61. The four least significant bits of 16-byte-aligned address keys were constant, making them a poor fit for a number of hash-table buckets that was a power of two. The hash function k % 65,521 assigned 16-byte-aligned address keys to all 50,000 buckets. The avg_cmp was 1.00.

The structure of the string keys also affected their bucket distribution. Character sum used 41 buckets for the student ID strings, with an avg_cmp of 1,200.94. For item0 through item49999, it used 131 buckets and produced an avg_cmp of 798.25. FNV-1a produced an avg_cmp of 1.42 for the student ID strings and 1.44 for the item names.

The hash rate was not closely correlated with hash ranking. For student ID keys, FNV-1a had a score of 5.31 nanoseconds per key, and character sum had a score of 5.91 nanoseconds per key. These nanosecond hash scores for student ID keys differ from avg_cmp rankings. Limitations of this lab include use of a wall-clock timer on one computer to measure hash times, which may be influenced by other factors related to computer use and programming choices. These findings apply only to the key values tested and may not be generalized to other key values.